

Applied Project 3: Imitation Learning

Group: Emilia Roma
Federica Morini, Giulia Pascoletti, Mattia Prandi



Why Imitation Learning?

Reinforcement Learning often requires a large number of interactions with the environment and carefully designed reward functions. In many tasks, defining an appropriate reward can be difficult, and exploration may become inefficient or unsafe.

Imitation Learning addresses these limitations by learning directly from expert demonstrations. Instead of discovering a behavior entirely through trial and error, the agent learns to reproduce trajectories generated by an expert policy.

Overview

Project Objective

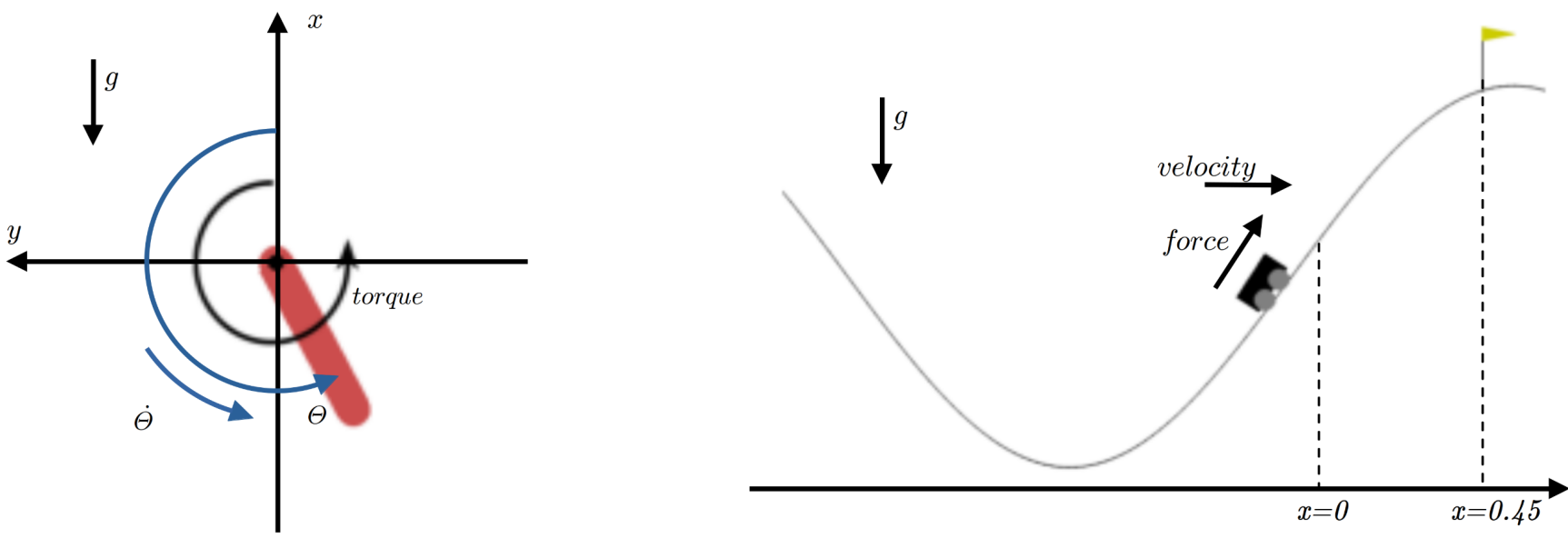
The goal of this project is to compare different imitation learning algorithms under varying amounts of expert demonstration data.

Train PPO Expert → Create Expert Dataset → Train IL Algorithms → Performance Evaluation

Selected Environments

Two Gymnasium [10] environments are considered:

- **Pendulum-v1**: continuous torque control task where the agent must swing up and stabilize a pendulum in the upright position.
- **MountainCarContinuous-v0**: continuous-control environment in which an underpowered car must learn to build momentum to reach the goal.



Expert Policy Generation

A PPO [4] agent (Stable-Baselines3 [9] implementation) is first trained to solve each environment. The resulting expert policy is then used to generate the trajectories composing the expert dataset used for imitation learning. Different expert dataset sizes are considered in order to analyse performance changes.

Algorithms and architectures

All three algorithms use the same SAC (Soft Actor-Critic) [5] backbone. The actor is a stochastic Gaussian policy. The sampled action is then squashed through a $\tanh$ function to keep it within the valid action range.

To estimate action values, the algorithms use two Q-networks (double Q-critics), which help improve training stability and reduce overestimation.

All neural networks are implemented in PyTorch [8] as MLPs with 2 hidden layers and 256 units per layer. Q-networks and actor use ReLU activations; the reward network and discriminator use $\tanh$ activations.

IQ-Learn [1] – inverse soft-Q learning.

- Learns a single $Q_\theta(s, a)$ jointly on expert and online transitions.
- Implicit reward: $\hat{r}(s, a, s') = Q(s, a) - \gamma V(s')$, with $V(s) = \mathbb{E}_{a \sim \pi}[Q(s, a) - \alpha \log \pi(a|s)]$.
- Uses a single Bellman-based objective to train the critic directly. The algorithm is trained off-policy using both expert and replay-buffer transitions.

f-IRL [2] – distribution-matching IRL.

- Three networks: a reward function bounded through a $\tanh$ function $r_\psi(s)$, a GAN-style [7] binary state discriminator $D_\omega(s)$ (analogous to GAIL [6]), and the SAC actor.
- Minimizes reverse-KL between state densities: $\min_\pi D_{\text{KL}}(\rho_\pi \parallel \rho_E)$.
- Each outer iteration (150 for **Pendulum**, 50 for **MtCar**): SAC policy update (1000 steps) → policy rollout (10 trajectories) → discriminator update (20 steps) → reward update (5 steps).

f-IRL + SOAR [3] – optimistic exploration on top of f-IRL.

- Difference from f-IRL: the standard SAC double-Q critic is replaced by an ensemble of $N = 4$ critics.
- The actor is trained using an optimistic Q-value:

$$Q_{\text{SOAR}}(s, a) = \bar{Q}(s, a) + \beta \cdot \text{Std}(Q_1(s, a), \dots, Q_N(s, a)).$$

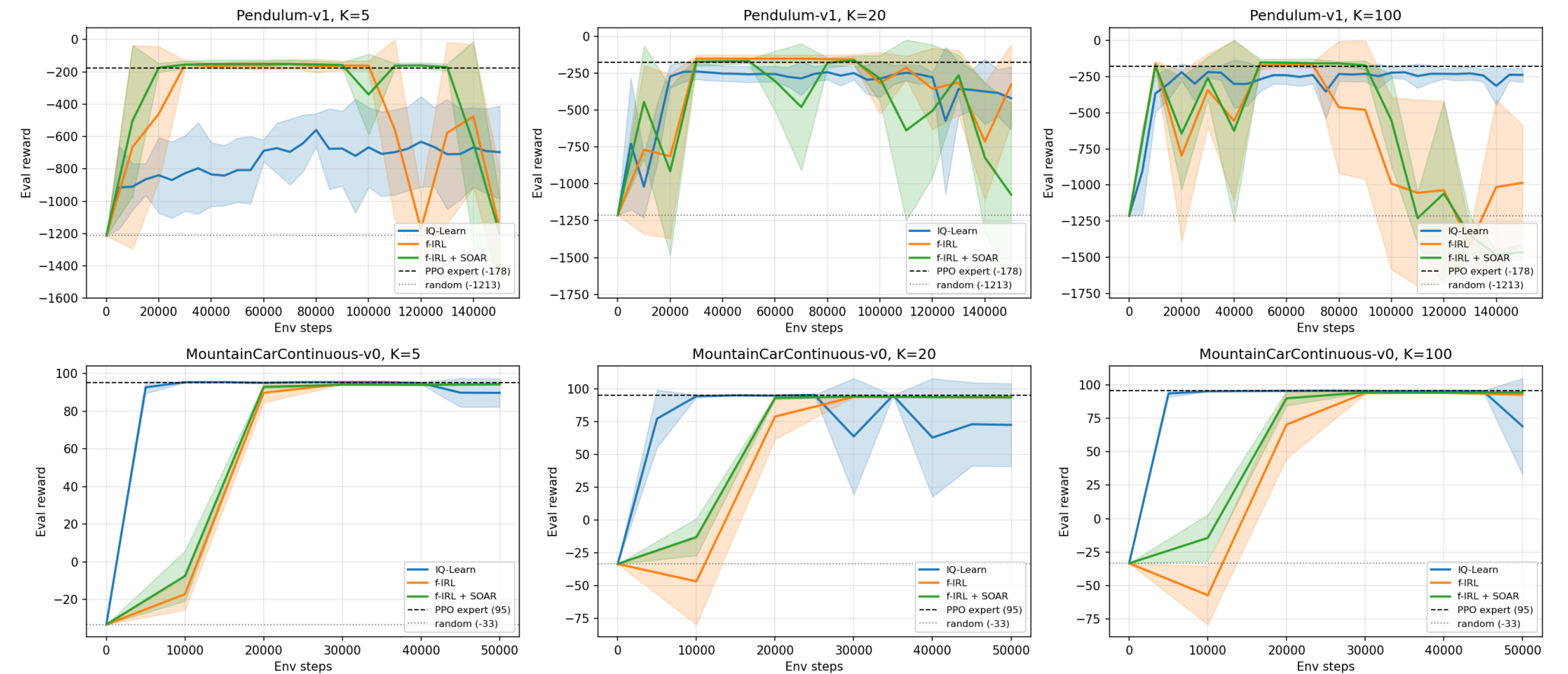
In our implementation, each ensemble member is a double-Q pair. For each pair we take the minimum critic value, and compute both $\bar{Q}$ and the standard deviation across these minima.

- The uncertainty term acts as an exploration bonus: actions with higher critic disagreement receive larger optimistic values and are explored more often.

Experimental setup

- **Dataset sizes**: for each environment, we evaluate expert datasets containing $K \in \{5, 20, 100\}$ trajectories, sampled from a fixed master dataset of 100 expert trajectories.
- **Budget**: 150,000 env steps for Pendulum-v1, 50,000 for MountainCarContinuous-v0.
- **Seeds**: experiments are repeated with seeds $\{0, 1, 2\}$. Reported curves show the mean performance ± 1 standard deviation across seeds.
- **Total**: 54 runs (3 algorithms $\times$ 2 environments $\times$ 3 dataset sizes $\times$ 3 seeds).
- **Baselines**: PPO expert (target) and uniform-random policy (lower bound).

Learning curves



Interpretation:

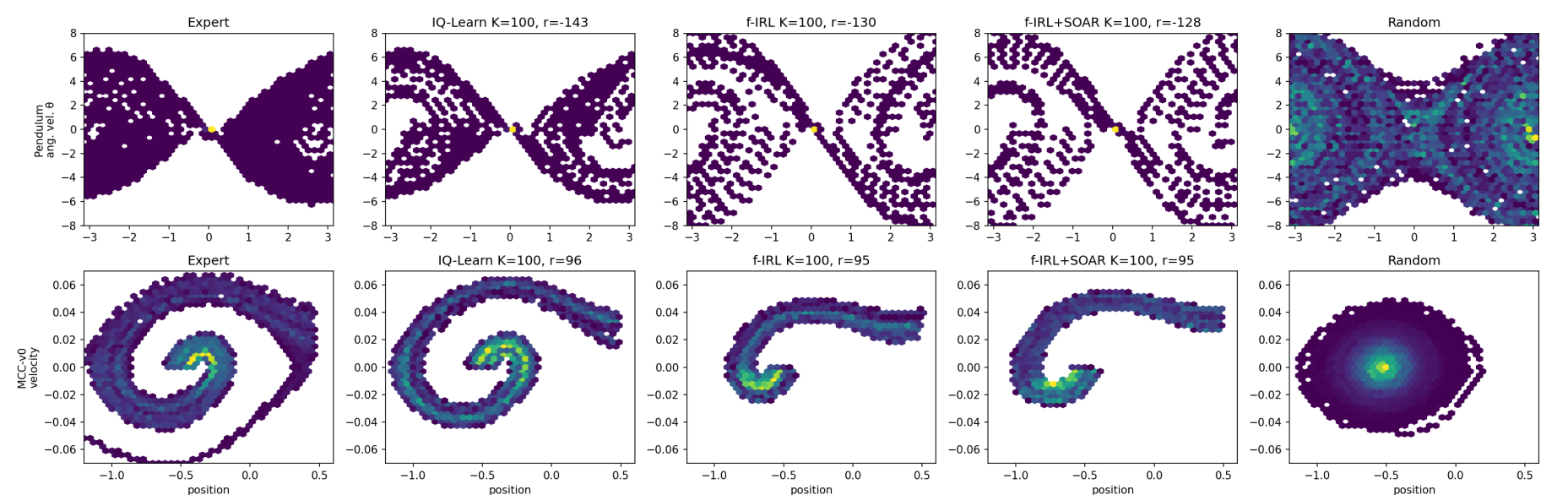
IQ-Learn (blue):

- **Pendulum**: performance improves with K , but at $K = 5$ the method remains unstable (≈ -560 reward with high variance). With very few expert samples, the χ^2 regularization provides insufficient guidance, leading to poor Q-function generalization. For $K = 20$ and $K = 100$, the method reaches near-expert performance (≈ -240 and -218); $K = 20$ shows a mild late decay likely due to slow critic drift, while $K = 100$ remains stable.
- **MtCar**: converges quickly across all K , reaching near-expert reward (≈ 95). However, a late-training decay is visible and grows with K : at $K = 5$ it is mild (final 89.8); at $K = 20$ the policy oscillates between 63 and 95 in the last 20k steps; at $K = 100$ a sharp drop occurs at the very last evaluation (final 68.9). This is likely because, once the agent consistently reaches the goal, many terminal transitions enter the replay buffer. For terminal states, the implicit reward reduces to the Q-value alone, creating a conflict between the Bellman update and the χ^2 regularization, which destabilizes the critic. Since larger K learns faster, this effect appears earlier and more strongly.

f-IRL and f-IRL + SOAR (orange/green):

- **Pendulum**: all K values exhibit a spike-and-collapse behaviour. The policy briefly reaches near-expert performance, but then rapidly degrades. Over training, the discriminator overfits the fixed expert dataset, its outputs saturate, and the covariance gradient used to update the reward network becomes too noisy to provide a reliable learning signal. SAC then optimizes this corrupted reward, leading to collapse. SOAR does not help: it adds exploration to the policy, but the failure originates upstream in the reward pipeline (discriminator saturation), which more exploration cannot fix.
- **MtCar**: both algorithms converge reliably to near-expert performance (≈ 94) within 20–30k steps for all K , with very low variance (final std < 0.7). Unlike IQ-Learn, f-IRL learns an explicit reward network, which provides a stable training signal even for terminal transitions and prevents the critic instabilities observed in IQ-Learn. SOAR brings no measurable benefit because, once the policy consistently reaches the goal, the ensemble critics become nearly identical and the disagreement drops toward zero, causing the exploration bonus to vanish.

State Visitation Density



Each panel shows the density of states visited over 30 deterministic episodes at $K=100$ (best checkpoint per algorithm). On **MtCar**, all three algorithms closely replicate the expert spiral trajectory, confirming successful goal-reaching behaviour. On **Pendulum**, all algorithms concentrate visits near the upright position, but with a broader spread compared to the expert.

Conclusions

- **IQ-Learn** is highly sample efficient and often reaches near expert performance rapidly, but its training can become unstable, especially with small datasets or long training horizons.
- **f-IRL** and **f-IRL+SOAR** reliably reproduce the expert state distribution on **MountainCarContinuous-v0**, but show unstable spike-and-collapse behaviour on **Pendulum-v1**.
- **SOAR** provides little improvement over f-IRL in our experiments. In both environments, the observed limitations lie outside what optimistic exploration can address.
- Overall, the results show that imitation learning performance depends strongly on the interaction between the algorithm, the environment dynamics, and the amount of expert data available.

References

- [1] Garg et al., *IQ-Learn: Inverse Soft-Q Learning for Imitation*, NeurIPS, 2021.
- [2] Ni et al., *f-IRL: Inverse Reinforcement Learning via State Marginal Matching*, CoRL, 2020.
- [3] Viel, Viano, and Cevher, *IL-SOAR: Imitation Learning with Soft Optimistic Actor Critic*, arXiv, 2025.
- [4] Schulman et al., *Proximal Policy Optimization Algorithms*, 2017.
- [5] Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, ICML, 2018.
- [6] Ho and Ermon, *Generative Adversarial Imitation Learning*, NeurIPS, 2016.
- [7] Goodfellow et al., *Generative Adversarial Nets*, NeurIPS, 2014.
- [8] Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.
- [9] Raffin et al., *Stable-Baselines3: Reliable Reinforcement Learning Implementations*, JMLR, 2021.
- [10] Towers et al., *Gymnasium: A Standard Interface for Reinforcement Learning Environments*, 2023.